EE2213 Introduction to Artificial Intelligence

Tutorial 5 (Lecture 6)

Dr. Shaojing Fan
fanshaojing@nus.edu.sg

Q1

Which of the following functions are convex? Select all that apply.

A: $f(x) = 5 - (x-2)^2, x \in \boldsymbol{\mathcal{R}}$

B: $f(x) = 7 - 3x, \ x \in \boldsymbol{\mathcal{R}}$

C: $f(x) = \log(x), x \in (0, +\infty)$

D: $f(x) = e^x, \ x \in \boldsymbol{\mathcal{R}}$

E: $f(x) = |2x + 2|, x \in \boldsymbol{\mathcal{R}}$

Why this question?

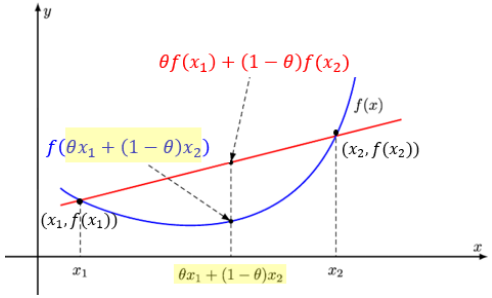
It's useful to know if a function is convex. Later on, especially in areas like machine learning, you'll need to design (objective) functions that can be minimized reliably. If the function is convex, you can be sure there's only one best solution, and your optimization method (e.g., gradient descent) will be able to find it.

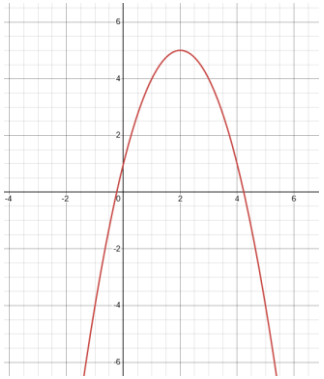
Q1 Method 1: Use the definition

Let C be a convex set. A function $f: C \to \mathbb{R}$ is convex in C if $\forall x_1, x_2 \in C$, $\forall \theta \in [0,1]$,

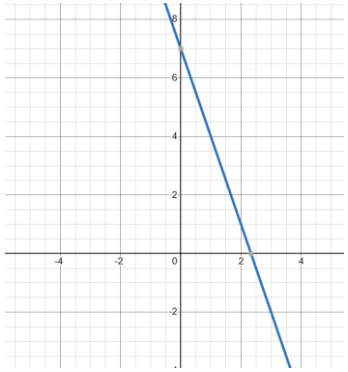
$$f(\theta x_1 + (1-\theta)x_2) \leq \theta f(x_1) + (1-\theta) f(x_2)$$

If $C = \mathbb{R}^n$, we simply say f is a convex function.

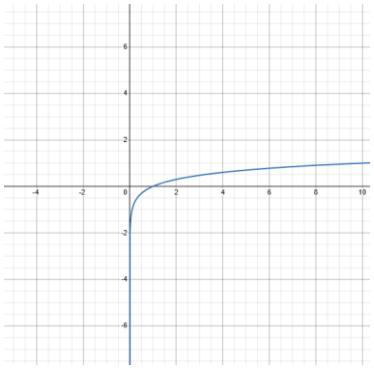




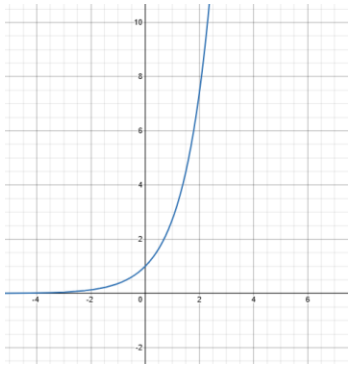
$f(x) = 5 - (x-2)^2$



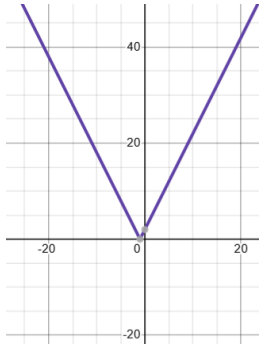
$f(x) = 7 - 3x$



$f(x) = \log(x)$



$f(x) = e^x$



$f(x) = |2x + 2|$

Q1

Which of the following functions are convex? Select all that apply.

A: $f(x) = 5 - (x-2)^2, x \in \boldsymbol{\mathcal{R}}$

B: $f(x) = 7 - 3x,\ x \in \boldsymbol{\mathcal{R}}$ ✓

C: $f(x) = \log(x), x \in (0, +\infty)$

D: $f(x) = e^x,\ x \in \boldsymbol{\mathcal{R}}$ ✓

E: $f(x) = |2x + 2|, x \in \boldsymbol{\mathcal{R}}$ ✓

Q1 Method 2: The second derivative test (for single-variable functions)

If the second derivative, $f''(x)$ is **always greater than or equal to 0** ($f''(x) \geq 0$) *throughout the function's domain*, then the function is **convex**, if $f''(x) \leq 0$, then the function is concave.

Example 1: For the function $f(x) = (x-2)^2$,
$f''(x) = 2$, which is always ≥ 0, so the function is convex.

Example 2: For the function $f(x) = \log(x),\ x \in (0, +\infty)$
$f'(x) = \frac{1}{x}$, $f''(x) = -\frac{1}{x^2}$, which is always < 0, so the function is concave.

Q2 True or false: For a convex optimization problem with a differentiable objective function, gradient descent always converges to the global minimum if the learning rate is chosen appropriately.

A. True

B. False

Q2 True or false: For a convex optimization problem with a differentiable objective function, gradient descent always converges to the global minimum if the learning rate is chosen appropriately.

A. True ✓

B. False

Q3

Why this question? **Gradient descent is one of the most common topics in machine learning interviews and a key concept behind many algorithms!**

Which of the following statements about gradient descent are correct? Select all that apply.

A: Gradient descent moves in the direction of steepest descent, which is the negative gradient of the function.

B: For convex functions with a properly chosen learning rate, gradient descent will converge to the global minimum.

C: If the learning rate is too large, gradient descent may overshoot and fail to converge.

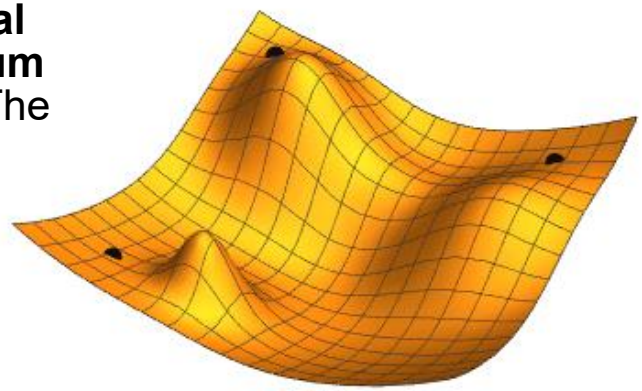
D: Gradient descent is guaranteed to find the global minimum even for non-convex functions.

E: Gradient descent only works for functions that are differentiable and strictly convex.

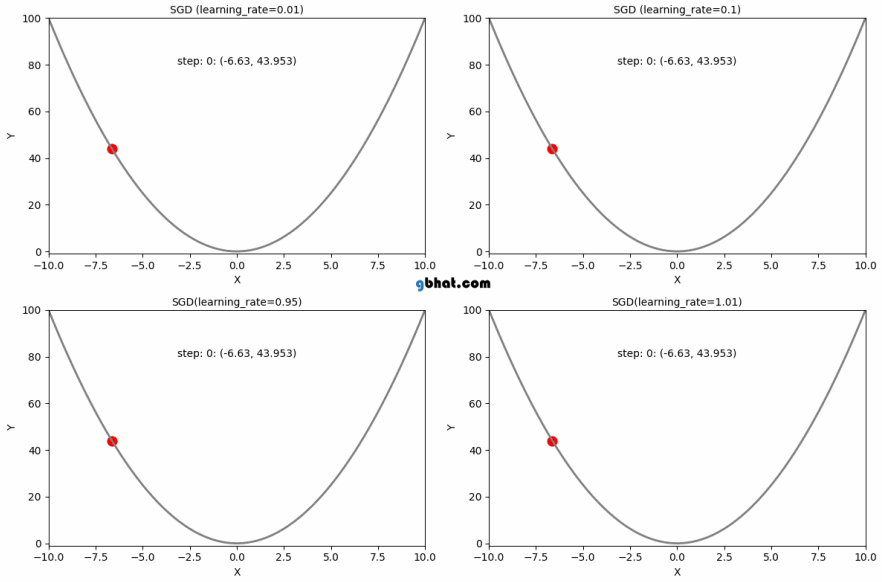
References: https://github.com/Devinterview-io/**gradient-descent-interview-questions**

Limitation of Gradient Descent

•**Gets stuck in local minima**: For *non-convex functions*, gradient descent can get stuck in a **local minimum**, and may fail to find the **global minimum** (the lowest possible point of the entire function). The starting point for the algorithm can significantly influence which minimum it converges to.



•**Initialization & learning rate sensitivity**: The outcome depends heavily on the starting point (initialization) and the learning rate (step size). A poor choice of either can lead to suboptimal results.



Source: https://commons.wikimedia.org/wiki/File:Gradient_descent.gif
https://gbhat.com/machine_learning/gradient_descent_learning_rates.html

Q3

Which of the following statements about gradient descent are correct? Select all that apply.

✓ A: Gradient descent moves in the direction of steepest descent, which is the negative gradient of the function.

✓ B: For convex functions with a properly chosen learning rate, gradient descent will converge to the global minimum.

✓ C: If the learning rate is too large, gradient descent may overshoot and fail to converge.

D: Gradient descent is guaranteed to find the global minimum even for non-convex functions.

E: Gradient descent only works for functions that are differentiable and strictly convex.

Q4

Suppose we are minimizing $f(x) = x^6$ with respect to x. We initialize $x = 1$, and perform gradient descent with learning rate $\eta = 0.01$.

(i) Calculate the updated value of x after the second iteration. Round your answer to 2 decimal places.

(ii) Explain why the updates in gradient descent become very small as x approaches 0. How does the shape of the function $f(x) = x^6$ influence the speed of convergence compared to a function like $f(x) = x^2$?

Q4 Common derivatives

[DERIVATION TABLES] --- Algebric and Logarithmes functions		
$\frac{d}{dx}(c) = 0$	$\frac{d}{dx}(cx) = c$	$\frac{d}{dx}(x^c) = cx^{c-1}$
$\frac{d}{dx}(c^x) = c^x \ln(c) \quad c > 0$	$\frac{d}{dx}(x^x) = x^x(1 + ln\, x)$	$\frac{d}{dx}(e^x) = e^x$
$\frac{d}{dx}\left(\frac{1}{x}\right) = \frac{-1}{x^2}$	$\frac{d}{dx}\left(\frac{1}{x^2}\right) = \frac{-2}{x^3}$	$\frac{d}{dx}\left(\frac{1}{x^n}\right) = \frac{-n}{x^{n+1}}$
$\frac{d}{dx}(\sqrt{x}) = \frac{1}{2\sqrt{x}} \quad x > 0$	$\frac{d}{dx}(\sqrt[3]{x}) = \frac{1}{3 \cdot \sqrt[3]{x^2}}$	$\frac{d}{dx}(\sqrt[n]{x}) = \frac{1}{n \cdot \sqrt[n]{x^{n-1}}}$
$\frac{d}{dx}\left(\frac{1}{\sqrt{x}}\right) = \frac{-1}{2\sqrt{x^3}}$	$\frac{d}{dx}\left(\frac{1}{\sqrt[3]{x}}\right) = \frac{-1}{3 \cdot \sqrt[3]{x^4}}$	$\frac{d}{dx}\left(\frac{1}{\sqrt[n]{x}}\right) = \frac{-1}{n \cdot \sqrt[n]{x^{n+1}}}$
$\frac{d}{dx}(\ln x) = \frac{1}{x} \quad x > 0$	$\frac{d}{dx}(x \cdot \ln x) = \ln x + 1$	$\frac{d}{dx}(\log_c x) = \frac{1}{x \ln c} \quad c > 0 \quad c \neq 1$
$\frac{d}{dx}\left(\frac{1}{lnx}\right) = \frac{-1}{x(\ln x)^2}$	$\frac{d}{dx}\left(\frac{1}{x \cdot \ln x}\right) = \frac{-(\ln x + 1)}{(x \cdot \ln x)^2}$	$\frac{d}{dx}\left(\frac{1}{\log_c x}\right) = \frac{-1}{x \cdot \ln c \cdot (\log_c x)^2}$
$\frac{d}{dx}\left(\frac{1}{x+1}\right) = \frac{-1}{(x+1)^2}$	$\frac{d}{dx}\left(\frac{1}{(x+1)^2}\right) = \frac{-2}{(x+1)^3}$	$\frac{d}{dx}\left(\frac{1}{(x+1)^n}\right) = \frac{-n}{(x+1)^{n+1}}$
$\frac{d}{dx}\left(\frac{1}{\sqrt{x+1}}\right) = \frac{-1}{2 \cdot \sqrt{(x+1)^3}}$	$\frac{d}{dx}\left(\frac{1}{\sqrt[3]{x+1}}\right) = \frac{-1}{3 \cdot \sqrt[3]{(x+1)^4}}$	$\frac{d}{dx}\left(\frac{1}{\sqrt[n]{x+1}}\right) = \frac{-1}{n \cdot \sqrt[n]{(x+1)^{n+1}}}$

Q4

Suppose we are minimizing $f(x) = x^6$ with respect to x. We initialize $x = 1$, and perform gradient descent with learning rate $\eta = 0.01$.
(i) Calculate the updated value of x after the second iteration.

1. Compute the gradient: $\nabla_x f(x) = 6x^5$
2. At each iteration, $x_{k+1} \leftarrow x_k - \eta \nabla_x f(x_k)$

Gradient Descent:
Initialize $\mathbf{x}_0$ and learning rate η
while true **do**
  Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$
  if converge **then**
    return $\mathbf{x}_{k+1}$
  end
end

Iteration 1

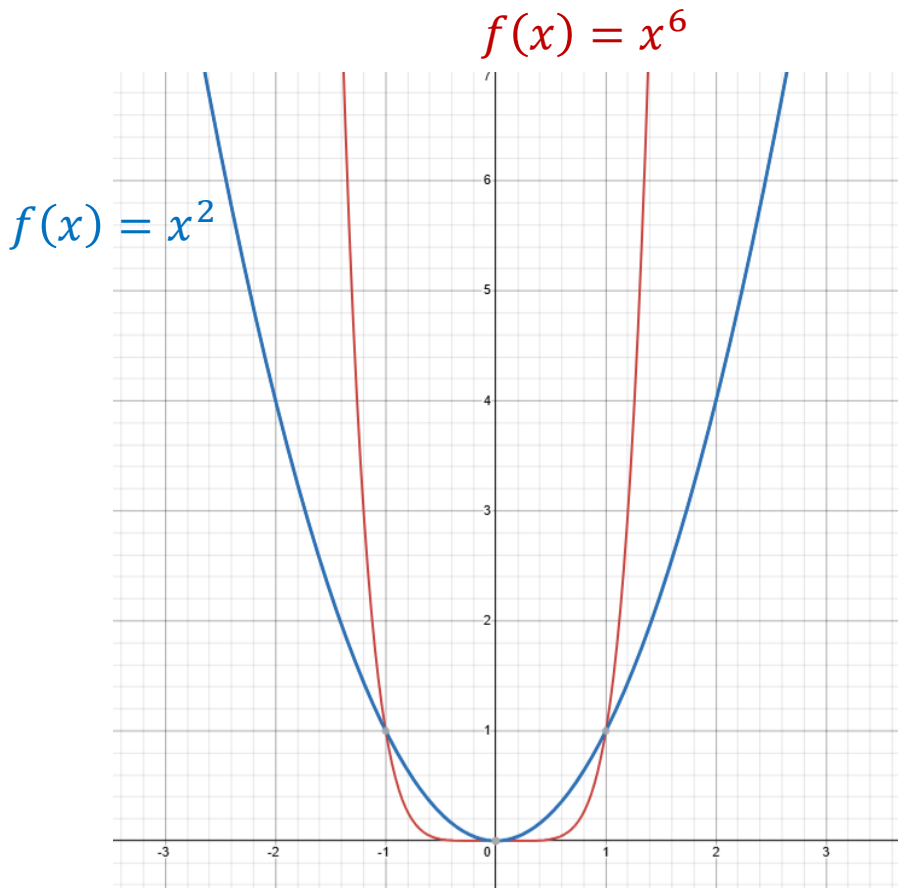
- $x_0 = 1$
- $\nabla_x f(x_0) = 6{x_0}^5 = 6$
- $x_1 = x_0 - 0.01 \nabla_x f(x_0) = 0.94$

Iteration 2

- $x_1 = 0.84$
- $\nabla_x f(x_1) = 6{x_1}^5 = 4.4034$
- $x_2 = x_1 - 0.01 \nabla_x f(x_1) = 0.90$ (round to 2 decimal places)

Q4

(ii) Explain why the updates in gradient descent become very small as x approaches 0. How does the shape of the function $f(x) = x^6$ influence the speed of convergence compared to a function like $f(x) = x^2$?



As x approaches 0, the gradient $f'(x) = 6x^5$ becomes very small, so the gradient descent updates shrink and progress slows down.

The shape of $f(x) = x^6$ is much flatter near $x = 0$, than $f(x) = x^2$, so convergence takes many more iterations.

Q5

Suppose we have a function that depends on a vector of numbers $\mathbf{w}$:

$$C(\mathbf{w}) = \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

where:

$\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m$ are given vectors,

$y_1, y_2, \ldots, y_m$ are given numbers (scalars).

Why this question?
Bridging to machine learning!

We want to minimize $C(\mathbf{w})$ using gradient descent.

(i) Check if the function $C(\mathbf{w})$ is convex. Explain your reasoning.

(ii) Compute the gradient of $C(\mathbf{w})$ with respect to $\mathbf{w}$ using $\sum$ notation.

(iii) Write the gradient descent update rule for $\mathbf{w}$ with learning rate η.

(iv) If $\mathbf{x}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $\mathbf{x}_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$, $\eta = 0.01$, we initialize $\mathbf{w} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, calculate the updated value of $\mathbf{w}$ after the first iteration.

(v) (Optional): Following the setup in (iv), solve the convex optimization problem using a solver (e.g., CVXPY).

Q5

Suppose we have a function that depends on a vector of numbers $\mathbf{w}$:

$$C(\mathbf{w}) = \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

where:

$\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m$ are given vectors,

$y_1, y_2, \ldots, y_m$ are given numbers (i.e., scalars).

We want to minimize $C(\mathbf{w})$ using gradient descent.

(i) Check if the function $C(\mathbf{w})$ is convex. Explain your reasoning.

Ans: Each term $(\mathbf{w}^T \mathbf{x}_i - y_i)^2$ is a squared linear function of $\mathbf{w}$. Squaring a linear function always gives a convex function. The sum of convex functions is also convex. Therefore, $C(\mathbf{w})$ is convex.

Q5

Suppose we have a function that depends on a vector of numbers $\mathbf{w}$:

$$C(\mathbf{w}) = \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

where:

$\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m$ are given vectors,

$y_1, y_2, \ldots, y_m$ are given numbers (i.e., scalars).

We want to minimize $C(\mathbf{w})$ using gradient descent.

(ii) Compute the gradient of $C(\mathbf{w})$ with respect to $\mathbf{w}$ using $\sum$ notation.

Ans: For a scalar function $f(\mathbf{w}) = (\mathbf{w}^T \mathbf{x} - y)^2$, the gradient with respect to $\mathbf{w}$ is:

$$\nabla_{\mathbf{w}} (\mathbf{w}^T \mathbf{x} - y)^2 = 2\ (\mathbf{w}^T \mathbf{x} - y)\mathbf{x}$$

The gradient of a sum is just the sum of the gradients:

$$\nabla_{\mathbf{w}} C(\mathbf{w}) = \nabla_{\mathbf{w}} \left[\sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i)^2 \right] = \sum_{i=1}^{m} \nabla_{\mathbf{w}} [(\mathbf{w}^T \mathbf{x} - y)^2] = 2 \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i)\mathbf{x}_i$$

Q5

Suppose we have a function that depends on a vector of numbers $\mathbf{w}$:

$$C(\mathbf{w}) = \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

where:

$\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m$ are given vectors,

$y_1, y_2, \ldots, y_m$ are given numbers (i.e., scalars).

We want to minimize $C(\mathbf{w})$ using gradient descent.

(iii) Write the gradient descent update rule for $\mathbf{w}$ with learning rate η.

$$\nabla_{\mathbf{w}} C(\mathbf{w}) = 2 \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i) \mathbf{x}_i$$

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}) = \mathbf{w}_k - 2\eta \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i) \mathbf{x}_i$$

Q5

(iv) Write the gradient descent update rule for $\mathbf{w}$ with learning rate η.

If $\mathbf{x}_1 = \begin{bmatrix}1\\0\end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix}0\\1\end{bmatrix}$, $\mathbf{x}_3 = \begin{bmatrix}1\\1\end{bmatrix}$, $\mathbf{y} = \begin{bmatrix}y_1\\y_2\\y_3\end{bmatrix} = \begin{bmatrix}2\\3\\5\end{bmatrix}$, $\eta = 0.01$, we initialize $\mathbf{w} = \begin{bmatrix}0\\0\end{bmatrix}$, calculate the updated value of $\mathbf{w}$ after the first iteration.

1. Compute the gradient: $\nabla_{\mathbf{w}} C(\mathbf{w}) = 2\sum_{i=1}^{3}(\mathbf{w}^T\mathbf{x}_i - y_i)\mathbf{x}_i$
2. At each iteration, $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta\, \nabla_{\mathbf{w}} C(\mathbf{w})$

Method 1: Solve by hand

Step 1: Compute gradient at $w_0 = \begin{bmatrix}0\\0\end{bmatrix}$

$i = 1, \mathbf{w_0}^T\mathbf{x}_1 - y_1 = 0 - 2 = -2, (\mathbf{w_0}^T\mathbf{x}_1 - y_1)\mathbf{x}_1 = \begin{bmatrix}-2\\0\end{bmatrix}$

$i = 2, \mathbf{w_0}^T\mathbf{x_2} - y_2 = 0 - 3 = -3, (\mathbf{w_0}^T\mathbf{x}_2 - y_2)\mathbf{x}_2 = \begin{bmatrix}0\\-3\end{bmatrix}$

$i = 3, \mathbf{w_0}^T\mathbf{x_3} - y_3 = 0 - 5 = -5, (\mathbf{w_0}^T\mathbf{x}_3 - y_3)\mathbf{x}_3 = \begin{bmatrix}-5\\-5\end{bmatrix}$

Sum:

$$\nabla_{\mathbf{w}} C(\mathbf{w_0}) = 2\sum_{i=1}^{3}(\mathbf{w_0}^T\mathbf{x}_i - y_i)\mathbf{x}_i = 2\left(\begin{bmatrix}-2\\0\end{bmatrix} + \begin{bmatrix}0\\-3\end{bmatrix} + \begin{bmatrix}-5\\-5\end{bmatrix}\right) = \begin{bmatrix}-14\\-16\end{bmatrix}$$

Step 2: Update w

$$\mathbf{w}_1 = \mathbf{w}_0 - \eta\, \nabla_{\mathbf{w}} C(\mathbf{w_0}) = \begin{bmatrix}0\\0\end{bmatrix} - 0.01\begin{bmatrix}-14\\-16\end{bmatrix} = \begin{bmatrix}0.14\\0.16\end{bmatrix}$$

Q5

(iv) Write the gradient descent update rule for $\mathbf{w}$ with learning rate η. If

$\mathbf{x}_1 = \begin{bmatrix}1\\0\end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix}0\\1\end{bmatrix}$, $\mathbf{x}_3 = \begin{bmatrix}1\\1\end{bmatrix}$, $\mathbf{y} = \begin{bmatrix}y_1\\y_2\\y_3\end{bmatrix} = \begin{bmatrix}2\\3\\5\end{bmatrix}$, $\eta = 0.01$, we initialize $\mathbf{w} = \begin{bmatrix}0\\0\end{bmatrix}$, calculate the updated value of $\mathbf{w}$ after the first iteration.

Method 2: Solve by Python

```python
import numpy as np

# Given data
X = np.array([[1, 0],
              [0, 1],
              [1, 1]])
y = np.array([[2], [3], [5]])
w = np.array([[0], [0]])  # initial w
eta = 0.01  # learning rate

# Compute the gradient
grad = 2 * X.T @ (X @ w - y)

# Update w
w_new = w - eta * grad

print("\nGradient (∇C(w)):", grad)
print("\nUpdated w after the first iteration:", w_new)
```

Initialize Gradient Descent
- Set X and y values
- Initialize w to be [0, 0]
- Set learning rate to 0.01

$$\nabla_{\mathbf{w}} C(\mathbf{w}) = 2\sum_{i=1}^{m}(\mathbf{w}^T\mathbf{x}_i - y_i)\mathbf{x}_i$$

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta\nabla_x f(\mathbf{x}_k)$$

Perform Gradient Descent
- Compute gradient
- Update w
  $w_1 \leftarrow w_0$ - learning_rate * gradient.

```
Gradient (∇C(w)): [[-14]
 [-16]]

Updated w after the first iteration: [[0.14]
 [0.16]]
```

Q5 Code highlights (for those new to Python)

import numpy as np

Import the NumPy library in Python, and assign it an alias “np”, so we can refer to it more easily later. NumPy is a powerful library for numerical computing in Python, especially useful for working with arrays, matrices, and mathematical functions.

X @ w

Multiply matrix X with vector w

The @ operator in NumPy performs matrix multiplication, different from element-wise multiplication (∗).

In our example: $\boldsymbol{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $\mathbf{w} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\mathrm{X} @ \mathrm{w} = \begin{bmatrix} 1*0+0*0 \\ 0*0+1*0 \\ 1*0+1*0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

X.T

Transpose of the matrix X, flips the rows and columns of X.

In our example: $\boldsymbol{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $\boldsymbol{X^T} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$

References: https://numpy.org/doc/stable/reference/index.html#reference

Q5 (Optional): Using CVXPY to solve the convex optimization problem

```python
import cvxpy as cp
import numpy as np

# Given data
X = np.array([[1, 0],
              [0, 1],
              [1, 1]])
y = np.array([[2],
              [3],
              [5]])

# Define variable
w = cp.Variable((2,1))  # w is a 2x1 column vector

# Define the objective function
objective = cp.Minimize(cp.sum_squares(X @ w - y))

# Define problem
problem = cp.Problem(objective)

# Solve
problem.solve()

# Results
print("\nOptimal weights w:")
print(w.value)
print("\nMinimum cost:", problem.value)
```

$$\min_{\mathbf{w}} C(\mathbf{w}) = \min_{\mathbf{w}} \sum_{i=1}^{m} (\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

Prepare data for objective function

Decision variables

Objective function

Create the convex optimization problem (w/o constraint)

Solve (typically non-gradient-based methods)

Get the result

```
Optimal weights w:
[[2.]
 [3.]]

Minimum cost: 4.930380657631324e-32
```

Q5 Takeaway: Gradient descent vs CVXPY

Non-convex problems

- CVXPY works only for convex problems.
- Many real-world ML tasks (e.g., training neural networks) are non-convex, and *gradient descent* (and its variants, e.g., SGD, mini-batch) can still be applied.

Large datasets and high-dimensional models

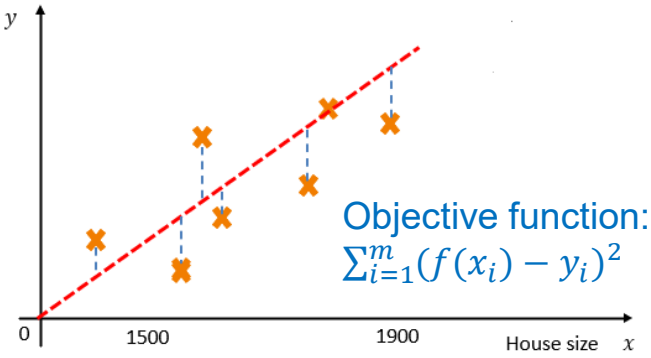
- *Gradient descent* and its variants (e.g., SGD, mini-batch) are scalable to huge datasets and high-dimensional parameter vectors.
- Convex solvers may be too slow or infeasible for large-scale problems.

Foundation for advanced algorithms

- *Gradient descent* provides the basis for understanding modern optimizers such as Adam and momentum methods.

Q5: Bridging to machine learning

1. Goal of machine learning in house price prediction:
Find the best fitting line to map from house size to price:
$y = ax + b$
a: slope, b: intercept



2. Pack parameters into a single vector: $\mathbf{w} = \begin{bmatrix} a \\ b \end{bmatrix}$

3. For each data point (x_i, y_i), define the feature vector to include a 1 for the intercept term:

$$\mathbf{x}_i = \begin{bmatrix} x_i \\ 1 \end{bmatrix}$$

Now the prediction is: $\mathbf{w}^T\mathbf{x}_i = a \cdot x_i + b \cdot 1$

4. We want to minimize the sum of squared error between predicted values ($\mathbf{w}^T\mathbf{x}_i$) and the true value (y_i):

$$C(\mathbf{w}) = \sum_{i=1}^{m} (\mathbf{w}^T\mathbf{x}_i - y_i)^2$$

$\mathbf{w}^T\mathbf{x}_i$: Prediction for sample i, i.e., predicted value
y_i: Actual (true) value
Squared error: (**prediction−truth**)²
$C(\mathbf{w})$ measures how well the line fits the data.